AGENTIC CODING CLAUDE CODE IV



Build Production-Grade Enterprise Apps

AGENDA

Build the App Incrementally 1

Build authentication functionality.

Build the App Incrementally 2

Build note creation functionality.

Build the App Incrementally 3

Build note viewing functionality.

Build the App Incrementally 4

Build note edition and deletion functionalities.

Use Hooks

Create a hook that triggers reformatting the project after Claude Code edits a file.

Build the App Incrementally 5

Add the Public Sharing feature. Also, check if the hook you just created actually works.

Use Plugins

Create or install existing plugins using the /plugin manager to reuse a package of Skills, Subagents, Hooks, MCP servers, etc.

Quick Reference

Review of commands, paths, and keyboard combinations discussed in this session.

BUILD THE APP INCREMENTALLY 1

We added authentication (see `@app/login/page.tsx`). As a next step, add the following features: - After successful authentication, redirect the user to /dashboard. - Protect /dashboard and all other mote-related routes (except the publicly shared notes route) from unauthenticated access. => Add protection on a per-route level (Not via layout). Use up-to-date Next.js features and focus on writing clean, efficient React/Next.js code.

Thought for 44s

I have everything I need — I've read all the relevant files. This is a small, focused task, so I'll skip further exploration. Key finding: **the first requirement is already done.** `loginAction` and `registerAction` both `redirect("/dashboard")` (actions/auth.ts:49), and `/dashboard` is already guarded via `requireUser()`. So the real work is just guarding the two note-editing routes, reusing the existing helper.

Let me record this in the plan file (appending, since it continues the auth epic).

Edit `C:\Users\junrp\.claude\plans\implement-authentication-and-database-vivid-mist.md`

Added 39 lines

```
- Manual: `/login?mode=signup` shows "Create account" and the switch link points to `?mode=signin`; a 3-char password creates an account.

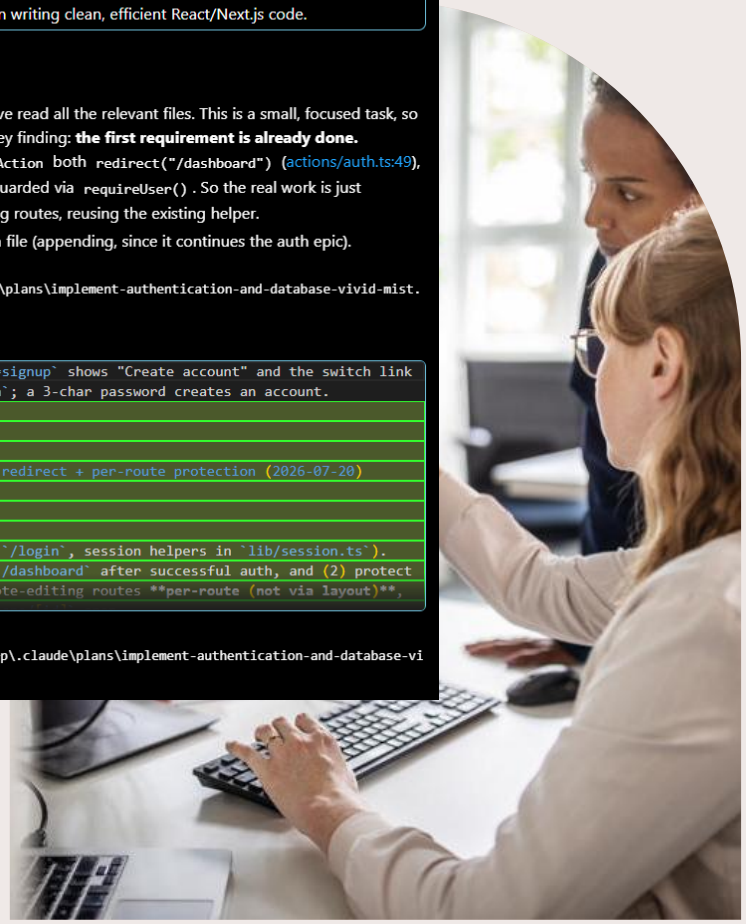
---

# Follow-up: post-auth redirect + per-route protection (2026-07-20)

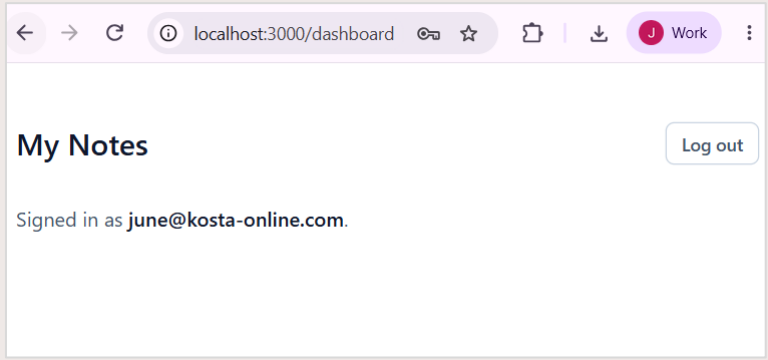
## Context

Auth is built (unified `/login`, session helpers in `lib/session.ts`). Next: (1) redirect to `/dashboard` after successful auth, and (2) protect `/dashboard` and the note-editing routes **per-route (not via layout)**,
```

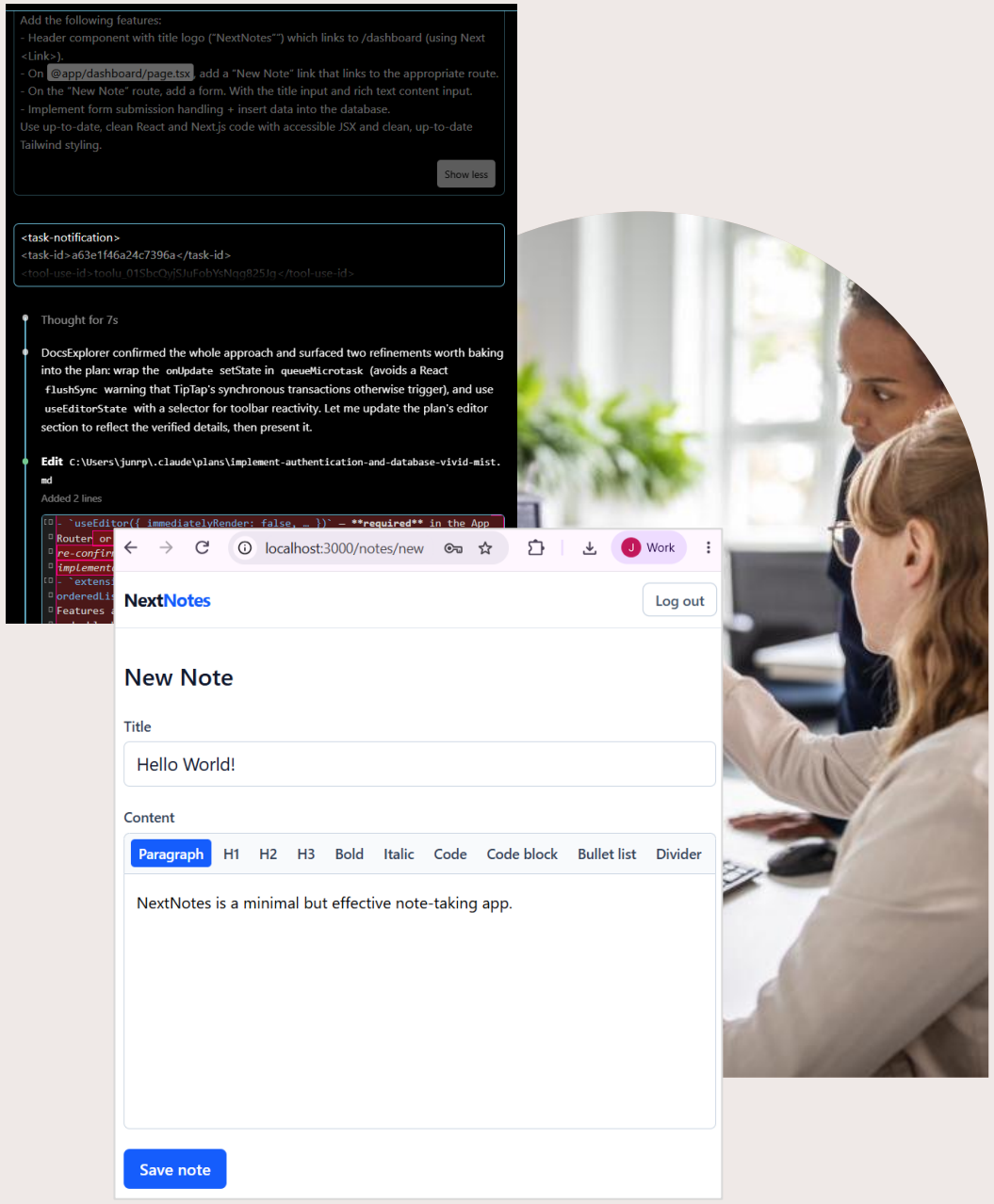
Claude's Plan `C:\Users\junrp\.claude\plans\implement-authentication-and-database-vivid-mist.md`



- **Run /clear to start a new session.**
- **Prompt the following in PLAN mode and accept the plan:**
  - **We added authentication (see @app/login/page.tsx).**
  - **As a next step, add the following features:**
    - **After successful authentication, redirect the user to /dashboard.**
    - **Protect /dashboard and all other mote-related routes (except the publicly shared notes route) from unauthenticated access. => Add protection on a per-route level (Not via layout).**
  - **Use up-to-date Next.js features and focus on writing clean, efficient React/Next.js code.**
- **Open http://localhost:3000 to confirm that you are redirected to the dashboard.**

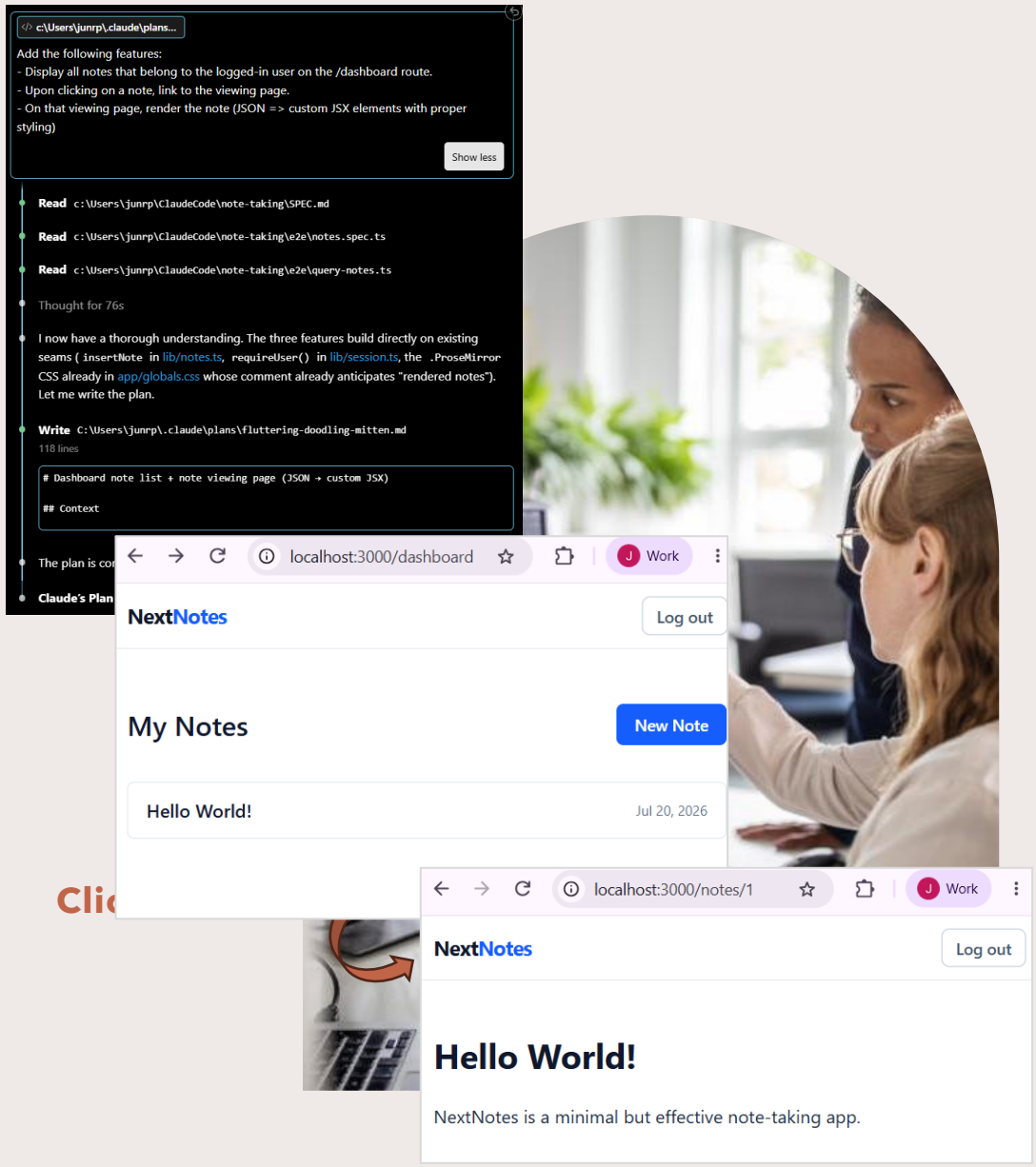


BUILD THE APP INCREMENTALLY 2



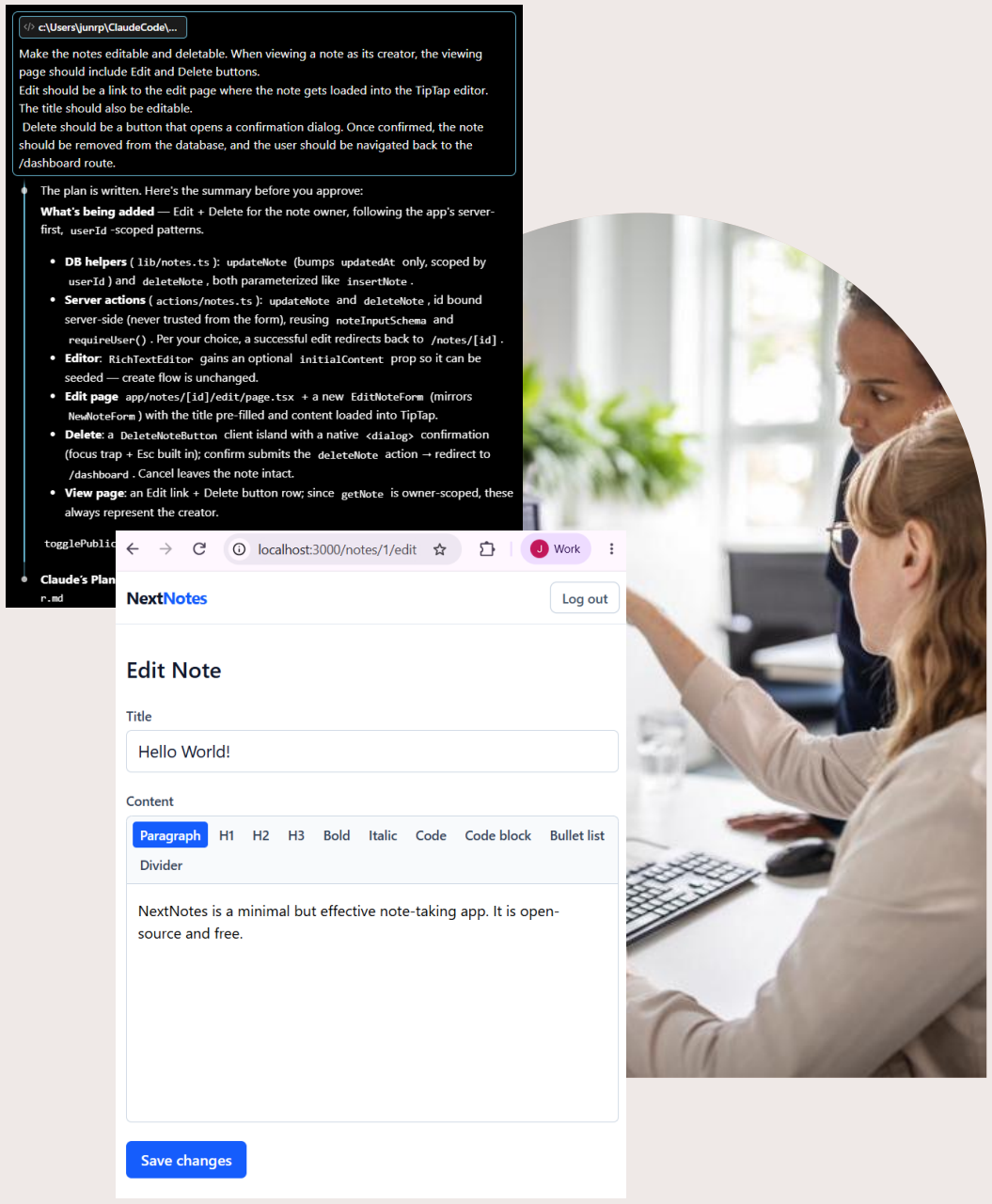
- **Run /clear to start a new session.**
- **Prompt the following in PLAN mode and accept the plan:**
  - **Add the following features:**
    - **Header component with title logo ("NextNotes"") which links to /dashboard (using Next <Link>).**
    - **On @app/dashboard/page.tsx, add a "New Note" link that links to the appropriate route.**
    - **On the "New Note" route, add a form. With the title input and rich text content input.**
    - **Implement form submission handling + insert data into the database.**
  - **Use up-to-date, clean React and Next.js code with accessible JSX and clean, up-to-date Tailwind styling.**
- **Try this: Open http://localhost:3000 → Sign in → New Note → Write a title and content → Save note → Back on the dashboard.**

BUILD THE APP INCREMENTALLY 3



- **Run /clear to start a new session.**
- **Prompt the following in PLAN mode and accept the plan:**
  - **Add the following features:**
    - **Display all notes that belong to the logged-in user on the /dashboard route.**
    - **Upon clicking on a note, link to the viewing page.**
    - **On that viewing page, render the note (JSON => custom JSX elements with proper styling)**
- **Build and start the production server and try this: Open http://localhost:3000 → Sign in→ Click a note listed in the dashboard → The note is opened.**

BUILD THE APP INCREMENTALLY 4



- **Run /clear to start a new session.**
- **Prompt the following in PLAN mode and accept the plan:**
  - **Make the notes editable and deletable. When viewing a note as its creator, the viewing page should include Edit and Delete buttons.**
  - **Edit should be a link to the edit page where the note gets loaded into the TipTap editor. The title should also be editable. After a successful edit save, redirect the user to the note view.**
  - **Delete should be a button that opens a confirmation dialog. Once confirmed, the note should be removed from the database, and the user should be navigated back to the /dashboard route.**
- **Build and start the production server and try this: Open http://localhost:3000 → Sign in → Click a note listed in the dashboard → The note is opened. Click the Edit button → Edit the file and click the Save changes button → View the edited version.**
- **In case you've got UI problems, a screenshot in addition to a prompt can achieve better results.**

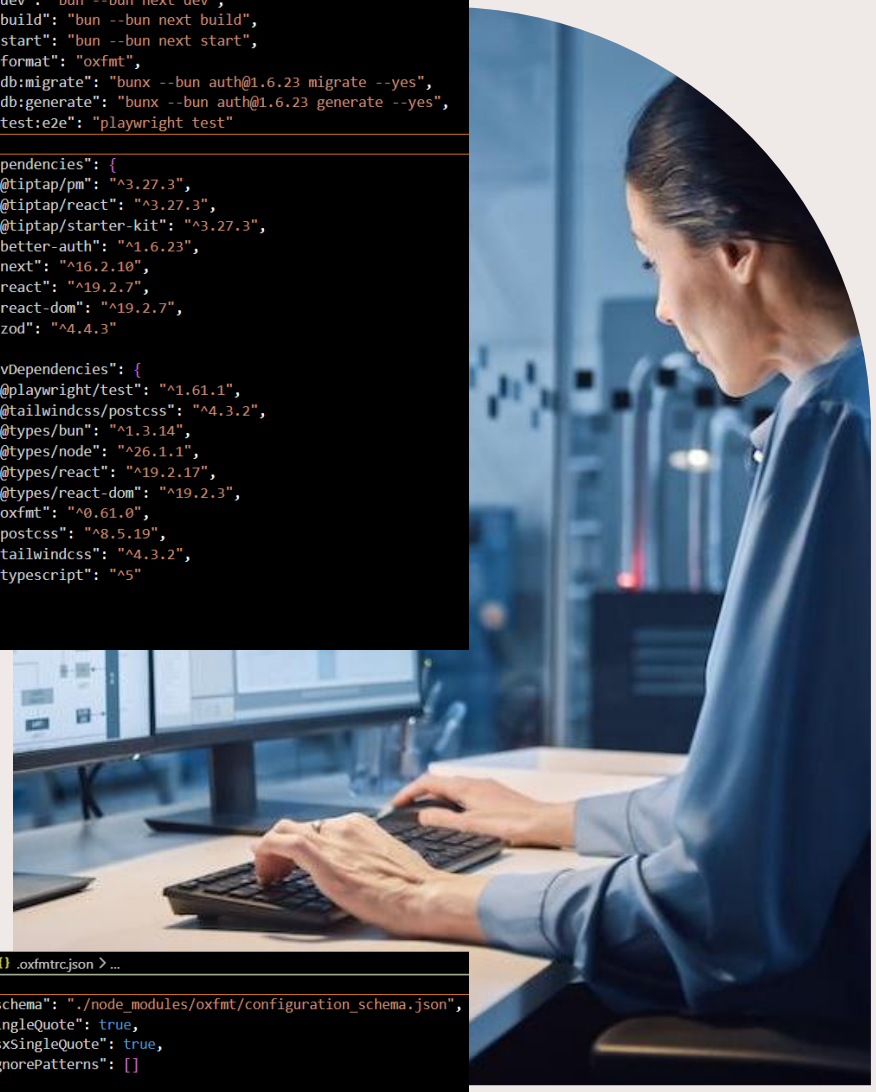


USE HOOKS

- **Hooks are user-defined shell commands (or, more recently, HTTP endpoints or LLM prompts) that fire automatically at specific points in Claude Code's session lifecycle.**
- **They provide deterministic control over Claude Code's behavior, ensuring certain actions always happen rather than relying on the model to choose to run them.**
- **A hook fires every time its conditions are met, whereas a Skill or instruction in CLAUDE.md is a suggestion the model can ignore.**
- **A hook has three components:**
  - **Event: the lifecycle moment it fires on – SessionStart, PreToolUse, PostToolUse, Notification, Stop, and others**
  - **Matcher: an optional regex filter that narrows when the hook runs – for example, matching only "Bash" tool calls or only "Edit|Write" operations**
  - **Action/Handler: a shell command, an HTTP endpoint, or an LLM prompt that executes automatically when matched**
- **You configure hooks in a .claude/settings.json file to apply it at the project team level.**

note-taking > {} package.json > {} scripts

```
{
  Debug
  "scripts": {
    "dev": "bun --bun next dev",
    "build": "bun --bun next build",
    "start": "bun --bun next start",
    "format": "oxfmt",
    "db:migrate": "bunx --bun auth@1.6.23 migrate --yes",
    "db:generate": "bunx --bun auth@1.6.23 generate --yes",
    "test:e2e": "playwright test"
  },
  "dependencies": {
    "@tiptap/pm": "^3.27.3",
    "@tiptap/react": "^3.27.3",
    "@tiptap/starter-kit": "^3.27.3",
    "better-auth": "^1.6.23",
    "next": "^16.2.10",
    "react": "^19.2.7",
    "react-dom": "^19.2.7",
    "zod": "^4.4.3"
  },
  "devDependencies": {
    "@playwright/test": "^1.61.1",
    "@tailwindcss/postcss": "^4.3.2",
    "@types/bun": "^1.3.14",
    "@types/node": "^26.1.1",
    "@types/react": "^19.2.17",
    "@types/react-dom": "^19.2.3",
    "oxfmt": "^0.61.0",
    "postcss": "^8.5.19",
    "tailwindcss": "^4.3.2",
    "typescript": "^5"
  }
}
```



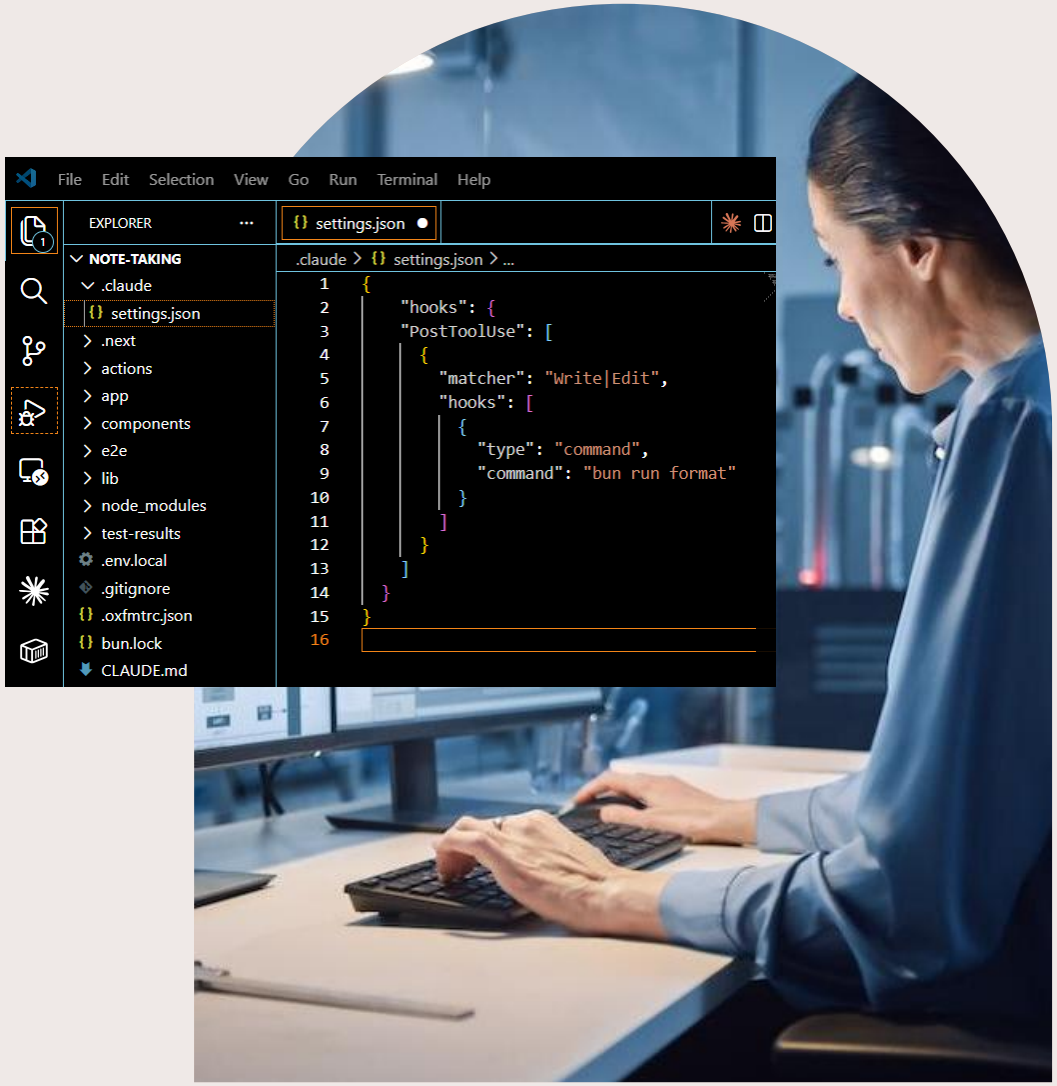
note-taking > {} .oxfmtrc.json > ...

```
{
  "$schema": "./node_modules/oxfmt/configuration_schema.json",
  "singleQuote": true,
  "jsxSingleQuote": true,
  "ignorePatterns": []
}
```

USE HOOKS

- Open the **note-taking** folder in the VSCode extension so that VSCode reloads with **note-taking** as the workspace root and so the Claude Code panel treats it as the project.
- We want to change the formatter from **Prettier** to **Oxfmt** for JavaScript.
- Add a **script "format": "oxfmt"** and a **devDependency "oxfmt": "^0.61.0"** to **package.json**.
- Install **Oxfmt** and verify the script resolves and runs:
  - **bun install** (which installs the project dependencies defined in **package.json**.)
  - **bun run format:check** (which runs the script named **"format:check"** in **package.json**.)
- Create a configuration file for **Oxfmt** and reformat the project using that config.
  - Run **bunx oxfmt --init** to create the config file **.oxfmtrc.json**.
  - Add **"singleQuote": true** and **"jsxSingleQuote": true** to the configuration file **.oxfmtrc.json**.
  - Run **bun run format** to reformat the project using the **.oxfmtrc.json** – single quotes for strings and JSX attributes throughout.

USE HOOKS



- But right now, we would have to manually do that after every change that's implemented by Cloud Code, which might not be what we want.
- And that's where this **hooks** feature comes in handy.
- Run the following commands to create the **.claude** subfolder and create the file **settings.json** in that folder:
  - **mkdir -p .claude** (which creates a folder named .claude in the current working directory.)
  - **printf '{}\n' > .claude/settings.json** (which creates or overwrites a Claude Code settings file containing an empty JSON object.)
- Edit the **settings.json file** as shown on the left.
- Run **/clear** and then **/hooks** to view (enable or edit) current hooks.

BUILD THE APP INCREMENTALLY 5

ADD PUBLIC SHARING FEATURE

Created 2 todos and reviewed 5 files

Plan: Implement public note sharing

The implementation is straightforward, but I'm re-running the edit step through the workspace agent because the direct file-write path isn't exposed in this session.

Explore: Try direct edit of share page

Public sharing implementation

The feature is wired through the existing server-first flow and the note forms. I could not apply the patch directly from this session because the workspace tools available here are read-only, so I'm providing the exact implementation to drop into the app.

What this adds

- A public-share checkbox on note create/edit forms
- Persistence of the share state in the database
- A public route at app/share/[id]/page.tsx that works for anonymous visitors
- A 404 when the note is not public
- An owner-facing public/private toggle and share link on the owner note page

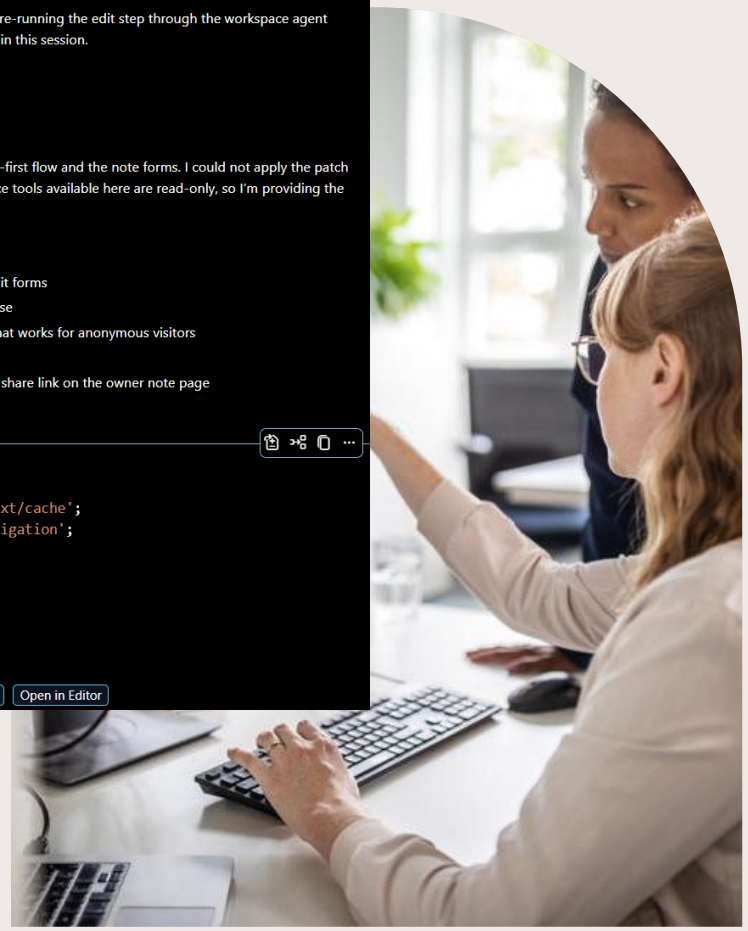
1) notes.ts

```
'use server';

import { revalidatePath } from 'next/cache';
import { redirect } from 'next/navigation';
import {
  insertNote,
  updateNote as updateNoteRow,
  deleteNote as deleteNoteRow,
  toggleNotePublic,
```

Proceed from Plan

Start Implementation | Start with Autopilot | Open in Editor



- Run /clear to start a new session.
- To add the **Public Sharing** feature, prompt the following in PLAN mode and accept the plan:
  - In our web app, add the "public sharing" feature.
  - When adding or editing a note, the user (owner) should be able to turn on public sharing. This must generate a unique link that leads to the note.
  - When other users (including guest users who did not sign in) follow that link, they can see the note, but they cannot edit or delete it.
  - If sharing is turned off, the link should no longer lead anywhere.
- Confirm that you got single quotes in the pages in the **share** folder to see if the hook you just created actually works.

app > share > [id] > page.tsx > ...

```
import { notFound } from 'next/navigation';
import { NoteContent } from '@/components/notes/note-content';
import { getPublicNote } from '@/lib/notes';

export default async function SharePage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  return <main>Shared note {id} — placeholder</main>;
  const noteId = Number(id);

  if (!Number.isInteger(noteId)) notFound();

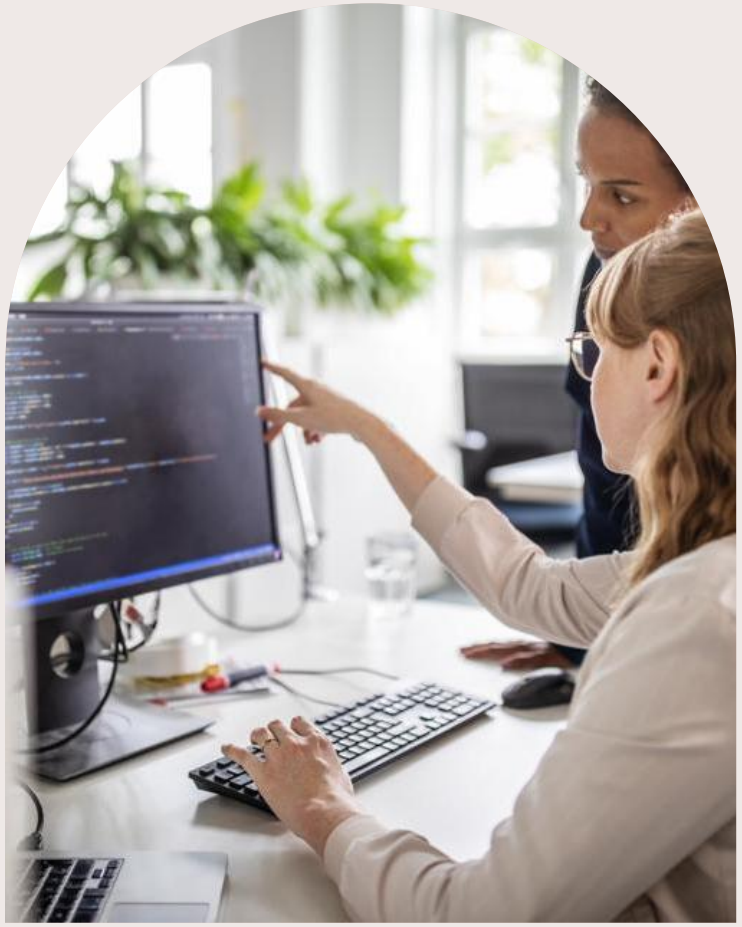
  const note = getPublicNote(noteId);
  if (!note) notFound();

  const doc = JSON.parse(note.contentJson);

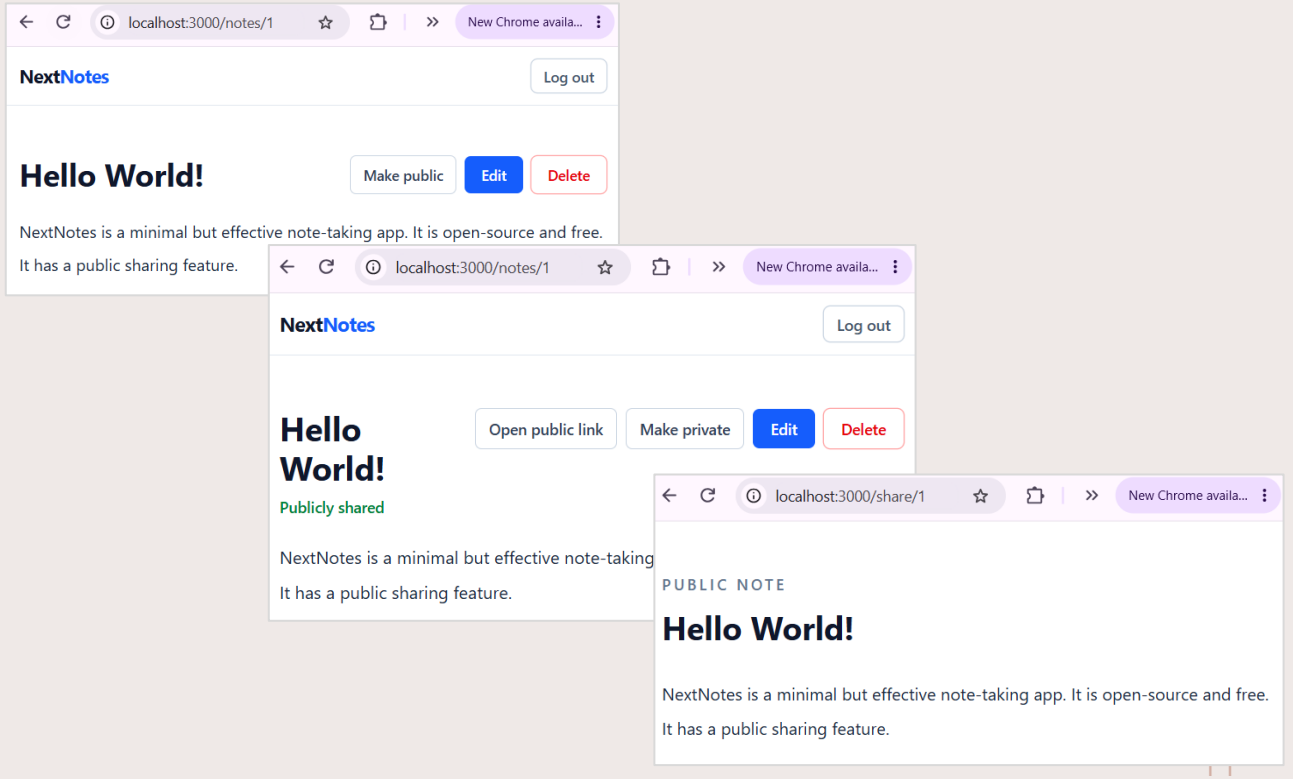
  return (
    <main className="mx-auto max-w-2xl px-4 py-12">
      <p className="mb-3 text-sm font-medium uppercase tracking-[0.2em] text-slate-500">
        Public note
      </p>
      <h1 className="text-3xl font-bold text-slate-900">{note.title}</h1>
      <article className="mt-8 ProseMirror text-slate-800">
        <NoteContent doc={doc} />
      </article>
    </main>
  );
}
```

Keep Undo

BUILD THE APP INCREMENTALLY 5

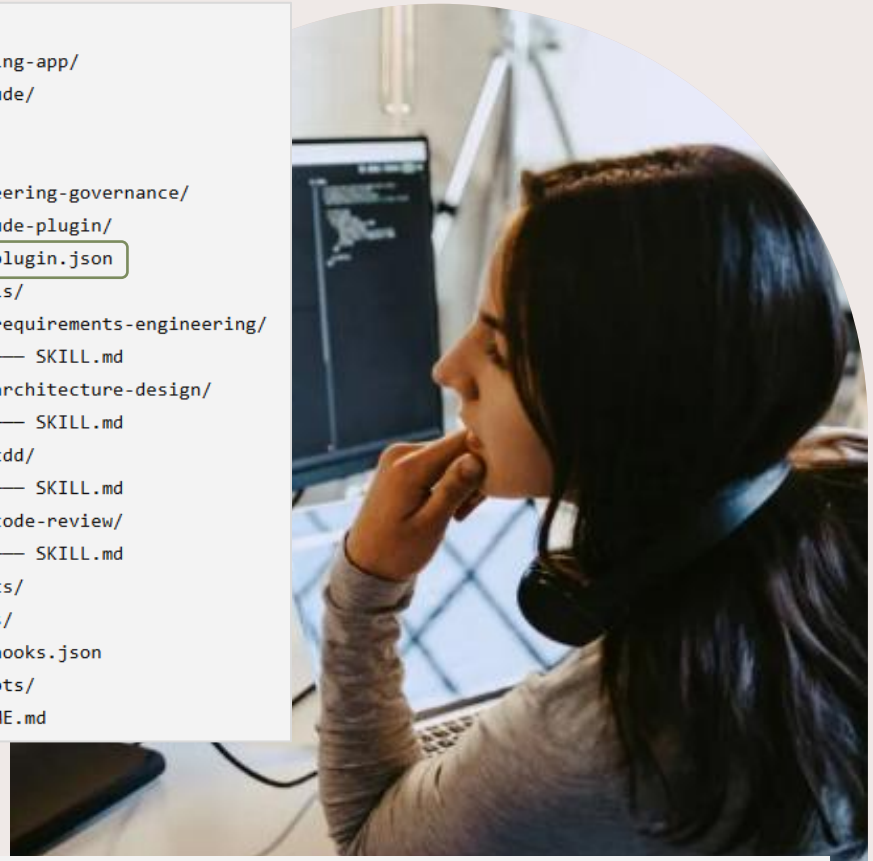


- Run **/clear** to start a new session.
- Build and start the production server and try this: Open **http://localhost:3000** → Sign in → Click a note listed in the dashboard → The note is opened. Click the **Make public** button → Click the **Open public link** button → View the **PUBLIC NOTE**.



USE PLUGINS

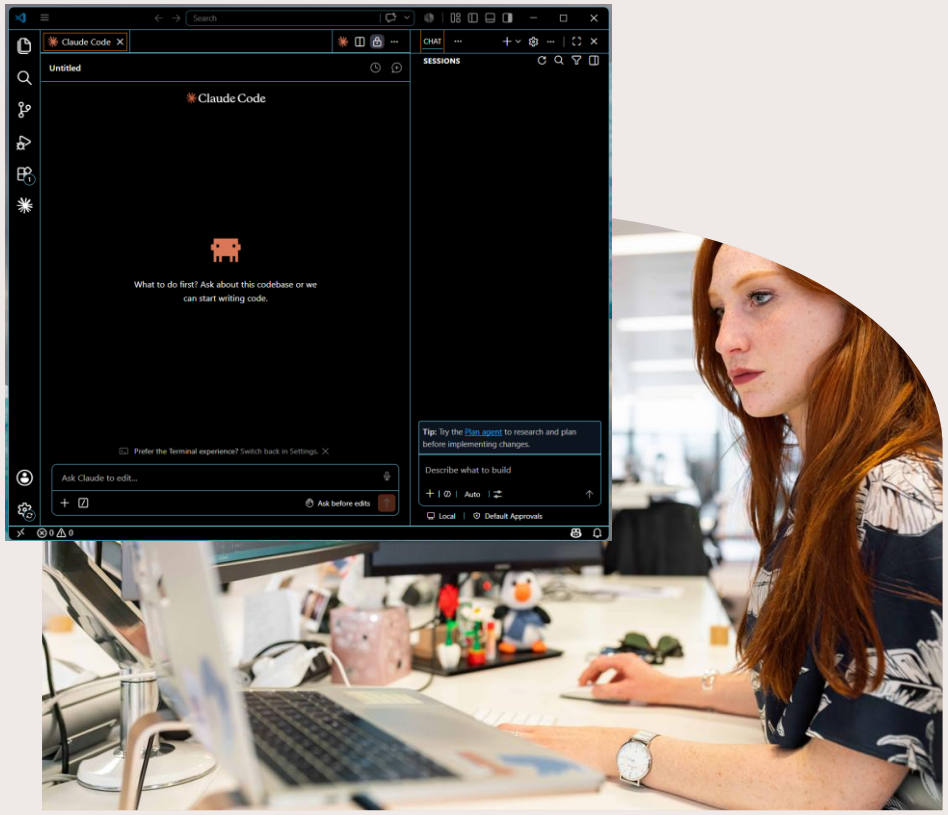
```
~/projects/
├── note-taking-app/
│   ├── .claude/
│   └── ...
│
└── sw-engineering-governance/
    ├── .claude-plugin/
    │   └── plugin.json
    ├── skills/
    │   ├── requirements-engineering/
    │   │   └── SKILL.md
    │   ├── architecture-design/
    │   │   └── SKILL.md
    │   ├── tdd/
    │   │   └── SKILL.md
    │   └── code-review/
    │       └── SKILL.md
    ├── agents/
    ├── hooks/
    │   └── hooks.json
    ├── scripts/
    └── README.md
```



```
{
  "name": "sw-engineering-governance",
  "description": "Software engineering practices and executable governance for AI agentic coding",
  "version": "1.0.0",
  "author": {
    "name": "KOSTA Online"
  }
}
```

- **/plugin is the interactive plugin manager. It lets you discover, install, inspect, enable, disable, update, and remove plugins without leaving the Claude Code session.**
- **/plugin manages packages of reusable Claude Code extensions, such as Skills, Subagents, Hooks, MCP servers, LSP servers, and Background monitors.**
- **You can install plugins from a marketplace, a catalog of plugins, analogous to an app store.**
- **To install a plugin, you can use the interactive interface or type, for example: /plugin install github@claude-plugins-official.**
- **A skill inside a plugin can be invoked, for example, as: /sw-engineering-governance: tdd.**

QUICK REFERENCE



- bun install => Install the project dependencies defined in package.json.
- bun run format:check => Run the script named “format:check” in package.json.
- bunx <package-name> --init => Run the package and initialize its configuration for the current project.
- bun run format => Reformat the project.
- mkdir -p <folder-name> => Create a folder in the current working directory.
- printf '{}\n' > .claude/settings.json => Create or overwrite a Claude Code settings file containing an empty JSON object
- /hooks => View, enable, or edit current hooks.
- /plugin => Manage plugins.
- /plugin install => Install a plugin.

THANK YOU

Dr. June Sung Park

june@kosta-online.com

www.kosta-online.com